Experiment 9

Ex. No.: 9

Hiding and Extracting a Text File Behind an Image or Audio File Using Command Prompt

Aim

To study binary file concatenation by appending a text or archive payload to an image or audio carrier and then recovering the appended data.

Tools required

1. Windows Command Prompt for the manual `copy /b` workflow.
2. A JPEG or audio carrier and a harmless text or ZIP payload.
3. macOS equivalents used here: `cat`, `tail`, `file`, `xxd`, `cmp`, and `shasum`.

Idea/Theory

1. Binary concatenation places the payload bytes after the carrier bytes; many viewers stop at the carrier end marker and still display the image or play the audio.
2. The appended bytes are not encrypted and can be found with a hex viewer, strings search, size analysis, or carving.
3. Keeping the carrier first helps the combined file remain recognisable to its normal viewer.
4. Hash comparison can verify that the extracted payload matches the original.

Experiment 9 — continued

Procedure/Steps

1. Place `A.txt` and `B.jpg` in the working directory.
2. In Windows Command Prompt, change to that directory and create the combined image:

```
cd Desktop
copy /b B.jpg + A.txt C.jpg
```

1. Open `C.jpg` in a text or hex viewer and inspect the bytes after the JPEG end marker.
2. To append a short message directly, the manual also uses:

```
echo "Your Message">>"image.jpg"
```

1. For an archive behind an audio carrier, keep the audio first and append the ZIP. On Linux or macOS:

```
cat sound.mp3 secret.zip > newfile.mp3
```

> This method is concealment only. It does not provide confidentiality or evidential authenticity.

Experiment 9 — continued

Procedure/Steps (executed sample)

The synthetic JPEG, secret text, combined JPEG, and extracted text are stored under `evidence/exp09/`.

```
cat carrier.jpg secret.txt > combined.jpg
tail -c "$(wc -c < secret.txt)" combined.jpg > extracted_secret.txt
file carrier.jpg combined.jpg extracted_secret.txt
wc -c carrier.jpg secret.txt combined.jpg extracted_secret.txt
cmp secret.txt extracted_secret.txt
shasum -a 256 secret.txt extracted_secret.txt
xxd -g 1 -s -96 combined.jpg
```

Executed on macOS using cat, tail, file, xxd, cmp, and shasum on sample files created for this report (the manual's Windows tool: Command Prompt copy /b).

Observation/Output

```
carrier.jpg:          JPEG image data, JFIF standard 1.01, aspect ratio, density 72x72, segment length
16, Exif Standard: [TIFF image data, big-endian, direntries=1], baseline, precision 8, 640x360,
components 3
combined.jpg:         JPEG image data, JFIF standard 1.01, aspect ratio, density 72x72, segment length
16, Exif Standard: [TIFF image data, big-endian, direntries=1], baseline, precision 8, 640x360,
components 3
extracted_secret.txt: ASCII text

Byte counts:
   24253 carrier.jpg
      68 secret.txt
   24321 combined.jpg
      68 extracted_secret.txt
   48710 total

Comparison:
cmp: secret.txt and extracted_secret.txt are identical

SHA-256:
66997201c68635b3340dc8dc174322d3915b3d22bb6f4d3de1cb81c5dc884e83  secret.txt
66997201c68635b3340dc8dc174322d3915b3d22bb6f4d3de1cb81c5dc884e83  extracted_secret.txt

Tail of combined JPEG:
00005ea1: a0 02 8a 28 a0 02 8a 28 a0 02 8a 28 a0 02 8a 28  ...(...(...(...(
00005eb1: a0 02 8a 28 a0 02 8a 28 a0 0f ff d9 53 79 6e 74  ...(...(....Synt
00005ec1: 68 65 74 69 63 20 6c 61 62 20 73 65 63 72 65 74  hetic lab secret
00005ed1: 3a 20 61 70 70 65 6e 64 65 64 20 63 61 72 72 69  : appended carri
00005ee1: 65 72 20 64 61 74 61 20 72 65 63 6f 76 65 72 65  er data recovere
00005ef1: 64 20 73 75 63 63 65 73 73 66 75 6c 6c 79 2e 0a  d successfully..
```

Result

The real output showed that the combined file was still identified as a 640×360 JPEG, its size increased by 68 bytes, and the appended text appeared after the JPEG end marker. `cmp` and matching SHA-256 digests demonstrated that the extracted 68-byte text was identical to the original sample.